

# GeeksforGeeks

A computer science portal for geeks

## GeeksQuiz

- [Home](#)
- [Algorithms](#)
- [DS](#)
- [GATE](#)
- [Interview Corner](#)
- [Q&A](#)
- [C](#)
- [C++](#)
- [Java](#)
- [Books](#)
- [Contribute](#)
- [Contests](#)
- [Jobs](#)

[Array](#)

[Bit Magic](#)

[C/C++](#)

[Articles](#)

[GFacts](#)

[Linked List](#)

[MCQ](#)

[Misc](#)

[Output](#)

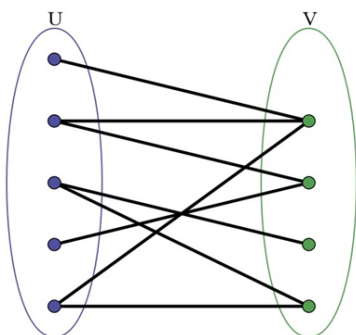
[String](#)

[Tree](#)

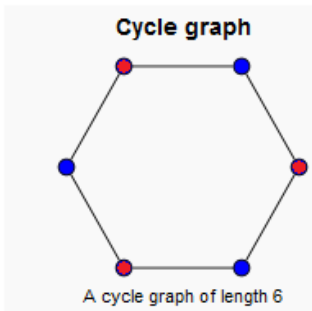
[Graph](#)

## Check whether a given graph is Bipartite or not

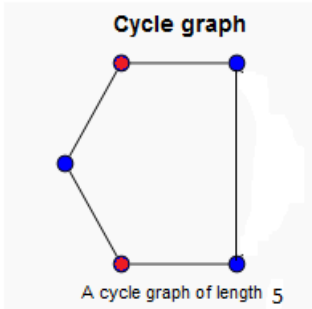
A [Bipartite Graph](#) is a graph whose vertices can be divided into two independent sets,  $U$  and  $V$  such that every edge  $(u, v)$  either connects a vertex from  $U$  to  $V$  or a vertex from  $V$  to  $U$ . In other words, for every edge  $(u, v)$ , either  $u$  belongs to  $U$  and  $v$  to  $V$ , or  $u$  belongs to  $V$  and  $v$  to  $U$ . We can also say that there is no edge that connects vertices of same set.



A bipartite graph is possible if the graph coloring is possible using two colors such that vertices in a set are colored with the same color. Note that it is possible to color a cycle graph with even cycle using two colors. For example, see the following graph.



It is not possible to color a cycle graph with odd cycle using two colors.



*Algorithm to check if a graph is Bipartite:*

One approach is to check whether the graph is 2-colorable or not using [backtracking algorithm m coloring problem](#).

Following is a simple algorithm to find out whether a given graph is Bipartite or not using Breadth First Search (BFS).

1. Assign RED color to the source vertex (putting into set U).
2. Color all the neighbors with BLUE color (putting into set V).
3. Color all neighbor's neighbor with RED color (putting into set U).
4. This way, assign color to all vertices such that it satisfies all the constraints of m way coloring problem where  $m = 2$ .
5. While assigning colors, if we find a neighbor which is colored with same color as current vertex, then the graph cannot be colored with 2 vertices (or graph is not Bipartite)

```
// C++ program to find out whether a given graph is Bipartite or not
```

```
#include <iostream>
```

```
#include <queue>
```

```
#define V 4
```

```
using namespace std;
```

```
// This function returns true if graph G[V][V] is Bipartite, else false
```

```
bool isBipartite(int G[][V], int src)
```

```
{
```

```
    // Create a color array to store colors assigned to all vertices. Vertex
    // number is used as index in this array. The value '-1' of colorArr[i]
    // is used to indicate that no color is assigned to vertex 'i'. The value
    // 1 is used to indicate first color is assigned and value 0 indicates
    // second color is assigned.
```

```
    int colorArr[V];
```

```
    for (int i = 0; i < V; ++i)
```

```
        colorArr[i] = -1;
```

```
    // Assign first color to source
```

```
    colorArr[src] = 1;
```

```

// Create a queue (FIFO) of vertex numbers and enqueue source vertex
// for BFS traversal
queue <int> q;
q.push(src);

// Run while there are vertices in queue (Similar to BFS)
while (!q.empty())
{
    // Dequeue a vertex from queue ( Refer http://goo.gl/35oz8 )
    int u = q.front();
    q.pop();

    // Find all non-colored adjacent vertices
    for (int v = 0; v < V; ++v)
    {
        // An edge from u to v exists and destination v is not colored
        if (G[u][v] && colorArr[v] == -1)
        {
            // Assign alternate color to this adjacent v of u
            colorArr[v] = 1 - colorArr[u];
            q.push(v);
        }

        // An edge from u to v exists and destination v is colored with
        // same color as u
        else if (G[u][v] && colorArr[v] == colorArr[u])
            return false;
    }
}

// If we reach here, then all adjacent vertices can be colored with
// alternate color
return true;
}

// Driver program to test above function
int main()
{
    int G[][V] = {{0, 1, 0, 1},
                  {1, 0, 1, 0},
                  {0, 1, 0, 1},
                  {1, 0, 1, 0}};

    isBipartite(G, 0) ? cout << "Yes" : cout << "No";
    return 0;
}

```

Output:

Yes

Refer [this](#) for C implementation of the same.

Time Complexity of the above approach is same as that Breadth First Search. In above implementation is  $O(V^2)$  where  $V$  is number of vertices. If graph is represented using adjacency list, then the complexity becomes  $O(V+E)$ .

### Exercise:

1. Can DFS algorithm be used to check the bipartite-ness of a graph? If yes, how?
2. The above algorithm works if the graph is strongly connected. Extend above code to work for graph with more than one component.

**References:**[http://en.wikipedia.org/wiki/Graph\\_coloring](http://en.wikipedia.org/wiki/Graph_coloring)[http://en.wikipedia.org/wiki/Bipartite\\_graph](http://en.wikipedia.org/wiki/Bipartite_graph)

This article is compiled by [Aashish Barnwal](#). Please write comments if you find anything incorrect, or you want to share more information about the topic discussed above

**Related Topics:**

- [Karger's algorithm for Minimum Cut | Set 1 \(Introduction and Implementation\)](#)
- [Greedy Algorithms | Set 9 \(Boruvka's algorithm\)](#)
- [Assign directions to edges so that the directed graph remains acyclic](#)
- [K Centers Problem | Set 1 \(Greedy Approximate Algorithm\)](#)
- [Find the minimum cost to reach destination using a train](#)
- [Applications of Breadth First Traversal](#)
- [Optimal read list for given number of days](#)
- [Print all paths from a given source to a destination](#)

Tags: [Graph](#)

Like   

**Writing code in comment?** Please use [ideone.com](http://ideone.com) and share the link here.

- *krooonal*

it checks only for 1 component ... for each component you need to make a query ... please correct that

- *Zheng Luo*

Thanks for sharing that.

- *Abhimanyu*

It is possible to test whether a graph is bipartite, and to return either a two-coloring (if it is bipartite) or an odd cycle (if it is not) in linear time, using depth-first search. The main idea is to assign to each vertex the color that differs from the color of its parent in the depth-first search tree, assigning colors in a preorder traversal of the depth-first-search tree. This will necessarily

provide a two-coloring of the spanning tree consisting of the edges connecting vertices to their parents, but it may not properly color some of the non-tree edges. In a depth-first search tree, one of the two endpoints of every non-tree edge is an ancestor of the other endpoint, and when the depth first search discovers an edge of this type it should check that these two vertices have different colors. If they do not, then the path in the tree from ancestor to descendant, together with the miscolored edge, form an odd cycle, which is returned from the algorithm together with the result that the graph is not bipartite. However, if the algorithm terminates without detecting an odd cycle of this type, then every edge must be properly colored, and the algorithm returns the coloring together with the result that the graph is bipartite.

- *Arpit Buddhiwant*

I think that the above given code is a solution to find whether 2-coloring of a given graph is possible or not.

But nothing can be said about the bipartite nature of the graph.

“A bipartite graph is possible if the graph coloring is possible using two colors.”

Its not “if and only if” condition and hence the vice versa is not necessarily true. The case with even cycle length serves as the example where this algorithm will produce incorrect results.

- *JOmegaCV*

Arpit – That is incorrect, it is an “if and only if”. A graph is bipartite if and only if it is two-colorable.

You can draw the even cycle graph as a bipartite graph if you put all the red nodes on one side, and all the blue on the other.

This is not true for the odd cycle graph, since two blue are connected.

- *Arpit Buddhiwant*

Got it. Thanks JOmegaCV.

- *sumit*

so how to handle if the graph is not SCC and has multiple components ?

- *pav*

I think this works for weakly connected graph too..

```
int[] colarr = new int[nVerts];
Queue q = new LinkedList();
boolean isBipartite(){
    for(int k = 0;k<colarr.length;k++){
        colarr[k] = -1;
    }
    for(int p=0;p<nVerts;p++){
        if (colarr[p]==-1) {
            boolean b = util(p);
            if(!b){
                return false;
            }
        }
    }
}
```

```

    }
    }
    return true;
}
private boolean xx(int po){
    q.add(po);
    colarr[po] = 1;
    while (!q.isEmpty()) {
        int c = q.remove();
        for(int i = 0; i < nVerts; i++){
            if(adjmat[i] == 1 && colarr[i] == -1){
                colarr[i] = (1 + colarr[c]) % 2;
                q.add(i);
            } else {
                if(adjmat[i] == 1 && colarr[i] == colarr[c]){
                    return false;
                }
            }
        }
    }
    return true;
}

```

#### ■ Guest

I think this works for weakly connected graph too..

```

int[] colarr = new int[nVerts];
Queue q = new LinkedList();
boolean isBipartite() {
    for(int k = 0; k

```

#### • coder

adjacency list representation and using dfs

```

#include<iostream>
#define v 9
#include<vector>
using namespace std;
class graph
{
    vector<int>*adj;
public:
    graph();
    void addedge(int x,int y);
    bool dfs(bool visited[],int color[],int current,int rang);
};
graph::graph()
{
    adj=new vector<int>[v];
}
void graph::addege(int x,int y)
{
    adj[x].push_back(y);
    adj[y].push_back(x);
}

```

```

bool graph::dfs(bool visited[],int color[],int current,int rang)
{
    visited[current]=true;
    vector<int>::iterator p;
    for(p=adj[current].begin();p!=adj[current].end();p++)
    {
        if(visited[*p])
        {
            if(color[*p]==color[current])
                return false;
        }
        else
        {
            if(rang==0)
            {
                color[*p]=1;
                if(dfs(visited,color,*p,1))
                    return true;
            }
            else
            {
                color[*p]=0;
                if(dfs(visited,color,*p,0))
                    return true;
            }
            return false;
        }
    }
    return true;
}

int main(void)
{
    graph g;
    g.addedge(0,5);
    g.addedge(1,5);
    g.addedge(1,6);
    g.addedge(2,7);
    g.addedge(2,8);
    g.addedge(3,6);
    g.addedge(4,5);
    g.addedge(4,8);
    int i,j,flag=0;
    int color[v];
    bool visited[v];
    for(i=0;i<=v-1;i++)
        visited[i]=false;
    for(i=0;i<=v-1;i++)
    {
        if(!visited[i])
        {
            color[i]=0;
            if(g.dfs(visited,color,i,0))
                continue;
            else
            {
                flag=1;
                break;
            }
        }
    }
    if(flag==0)
    {
        cout<<"the graph is bipartite"<<"\n";
        for(i=0;i<=v-1;i++)
            cout<<color[i]<<' ';
    }
}

```

```

        cout<<'\\n';
    }
    else
    cout<<"the graph contains an odd edged cycle"<<'\\n';
    return 0;
}

```

- *Vibhu Tiwari*

Nice one but can you please also post for the dfs.Bcz i think that it will be more efficient.

- *kartik*

How is DFS more efficient? Could you please explain?

- *Sachin*

Good article.

DFS can also be used

- 
- 



- 
- - [Interview Experiences](#)
  - [Advanced Data Structures](#)
  - [Dynamic Programming](#)
  - [Greedy Algorithms](#)
  - [Backtracking](#)
  - [Pattern Searching](#)
  - [Divide & Conquer](#)
  - [Mathematical Algorithms](#)
  - [Recursion](#)
  - [Geometric Algorithms](#)

- 

## • Popular Posts

- [All permutations of a given string](#)
- [Memory Layout of C Programs](#)
- [Understanding “extern” keyword in C](#)
- [Median of two sorted arrays](#)
- [Tree traversal without recursion and without stack!](#)
- [Structure Member Alignment, Padding and Data Packing](#)
- [Intersection point of two Linked Lists](#)
- [Lowest Common Ancestor in a BST.](#)
- [Check if a binary tree is BST or not](#)
- [Sorted Linked List to Balanced BST](#)

- 

## • Recent Comments

- [mishra](#)

few months ago,since i busy those days with my...

[Snapdeal Interview Experience | Set 14 \(On-Campus for Software Engineer 1\)](#) · [3 minutes ago](#)

- [tejavadali](#)

We can use Tries in place of hashmap to get...

[Find number of distinct palindromic sub-strings of a given string](#) · [17 minutes ago](#)

- [sampath g k](#)

its undirected graph ,where each edge is two...

[Detect cycle in an undirected graph](#) · [22 minutes ago](#)

- [Rajesh94](#)

Multiple strong symbols are not allowed. It...

[How Linkers Resolve Multiply Defined Global Symbols?](#) · [1 hour ago](#)

- [pk](#)

Just include `#include<algorithm>` The main logic...

[Write a C Program to Find the Maximum Depth or Height of a Tree](#) · [2 hours ago](#)

- [s3](#)

Hi bhaiya, when did this interview take place?...

[Snapdeal Interview Experience | Set 14 \(On-Campus for Software Engineer 1\)](#) · [3 hours ago](#)

•

@geeksforgeeks, [Some rights reserved](#) \_\_\_\_ [Contact Us!](#) \_\_\_\_ [Abut Us!](#)  
Powered by [WordPress](#) & [MooTools](#), customized by geeksforgeeks